Relazione di Progetto: Tetris su ARM Cortex-M3

Domenico di Stasio

1 – Panoramica del Sistema e Architettura Software

Il progetto consiste nell'implementazione *bare-metal* del videogioco Tetris sulla scheda di sviluppo **LandTiger (NXP LPC1768)**. L'applicazione è stata sviluppata in linguaggio C, interagendo direttamente con i registri delle periferiche tramite standard CMSIS, senza l'ausilio di sistemi operativi.

L'architettura software è di tipo **Event-Driven**: il main inizializza le periferiche e pone il processore in stato di risparmio energetico (__ASM("wfi")), delegando l'intera logica di esecuzione alle **ISR (Interrupt Service Routines)**.

Di seguito sono descritti i moduli funzionali principali e le strategie adottate per la gestione delle risorse.

1.1 Scheduling e Gestione del Tempo (RIT)

Il cuore pulsante del sistema è il **Repetitive Interrupt Timer (RIT)**. Configurato per generare un interrupt a intervalli regolari (tipicamente 50ms), la ISR del RIT funge da *scheduler* semplificato per i task periodici:

- **Polling degli input:** Scansione dello stato del Joystick per i movimenti laterali e la rotazione.
- **Logica di Gravità:** Decremento dei contatori per la discesa automatica dei tetromini.
- **Animazione:** Gestione delle tempistiche per il refresh del display.

1.2 Gestione Input e Debouncing

La gestione dell'interazione utente adotta un approccio ibrido per massimizzare la reattività e la stabilità:

- **Pulsanti (INT0, INT1, INT2):** Gestiti tramite interrupt esterni (EINT) per garantire una risposta immediata a eventi critici come *Start*, *Pause* e *Hard Drop*. Le priorità NVIC sono state configurate per dare precedenza ai comandi di gioco rispetto ad altri interrupt.
- **Joystick:** Gestito tramite **polling** all'interno della ISR del RIT. Questa scelta permette un controllo continuo e fluido del pezzo (movimento destra/sinistra) e rende il joystick più utilizzabile, evitando che i tetramini si muovano troppo velocemente.

1.3 Game Engine e Ottimizzazione Grafica

La logica di gioco risiede nel modulo tetris.c e si basa su una matrice 20x10 che rappresenta il campo di gioco (Board). Per garantire un frame-rate elevato su un display pilotato via bus parallelo, è stata implementata una strategia di **Rendering Parziale**:

1. Invece di ridisegnare l'intero schermo a ogni frame (operazione costosa in termini di cicli di clock), il sistema traccia la posizione precedente del tetramino (game.prev_current).
2. Ad ogni tick, viene "cancellata" solo la vecchia posizione (sovrascrivendola con il colore di sfondo o ridisegnando la griglia/pezzi adiacenti) e disegnata la nuova posizione.
3. Il refresh completo (LCD_Clear) avviene solo nelle transizioni di stato maggiori (es. Game Over o Reset).

1.4 Gestione Dinamica della Velocità (ADC)

Per implementare la specifica richiesta sulla regolazione della velocità di gioco, è stato utilizzato il convertitore Analogico-Digitale (**ADC**) collegato al potenziometro della scheda. Il valore letto viene filtrato via software tramite una funzione di *denoising* (ADC_CleanNoise) che scarta i bit meno significativi affetti da rumore. Il

risultato viene mappato sulla variabile gameSpeedDivider, permettendo all'utente di alterare la velocità di caduta dei pezzi in tempo reale.

1.5 Comparto Audio (Timer Hardware)

La generazione degli effetti sonori e della musica di sottofondo è gestita dal **Timer0**. Anziché utilizzare cicli di attesa bloccanti, il Timer0 è configurato per generare interrupt alla frequenza della nota corrente, effettuando il *toggle* di un pin collegato allo speaker. Questo permette alla musica di fluire in background senza interrompere la logica di gioco o la gestione degli input.

2 – Repetitive Interrupt Timer (RIT): Scheduling e Gestione Input

Il **Repetitive Interrupt Timer (RIT)** rappresenta il cuore pulsante dell'architettura *event-driven* del progetto. A differenza dei Timer general-purpose (Timer0, Timer1), il RIT è stato scelto per la sua specifica capacità di generare interrupt a intervalli fissi con un overhead di configurazione minimo, rendendolo ideale per lo scheduling di task periodici come il polling degli input e l'aggiornamento della fisica di gioco.

2.1 Configurazione e Timing

L'inizializzazione del timer avviene nel file lib_RIT.c. Il timer è configurato per operare con una frequenza di clock derivata dal PCLK (Peripheral Clock).

L'intervallo di interrupt è stato calcolato per ottenere una frequenza di aggiornamento di 50ms (20 Hz). Questo valore rappresenta un compromesso ottimale tra:

1. **Reattività:** Sufficiente per catturare gli input del joystick senza latenza percepibile.
2. **Efficienza:** Evita di sovraccaricare la CPU con interrupt troppo frequenti, lasciando tempo di calcolo per il rendering grafico sul display LCD.

La formula utilizzata per il calcolo del valore di comparazione (RICOMPVAL) è:

$$RIT_Count = T_{tick} \times f_{PCLK}$$

Dove T_{tick} = 50ms. All'attivazione dell'interrupt, il flag viene azzerato e il conteggio riparte automaticamente, garantendo una base dei tempi stabile e priva di deriva.

2.2 Architettura della Interrupt Service Routine (ISR)

La routine di gestione dell'interrupt (RIT_IRQHandler in IRQ_RIT.c) funge da **scheduler non bloccante**. Ad ogni "tick" del timer (50ms), vengono eseguite sequenzialmente le seguenti operazioni:

1. **Polling del Joystick:** Viene letto lo stato delle porte GPIO collegate al joystick (P1.25 - P1.29). A differenza dei pulsanti gestiti via EINT, il joystick viene gestito in polling per filtrare naturalmente i rimbalzi meccanici (*debouncing*): campionando lo stato ogni 50ms, eventuali segnali spuri transitori vengono ignorati.
2. **Mappatura Azioni:**
   - J_LEFT → Invoca Action_MoveLeft().
   - J_RIGHT → Invoca Action_MoveRight().
   - J_UP → Invoca Action_Rotate().
3. **Gestione della Gravità:** La caduta automatica dei tetramini non avviene ad ogni tick (sarebbe troppo veloce), ma è regolata da un contatore interno. Quando questo contatore raggiunge una soglia definita dalla variabile globale gameSpeedDivider (aggiornata dinamicamente dall'ADC), viene invocata la funzione Action_GravityTick().

2.3 Vantaggi dell'approccio basato su RIT

L'utilizzo del RIT per la gestione del joystick e della gravità offre vantaggi significativi rispetto a un approccio basato puramente su interrupt esterni (EINT) o su loop infiniti:

- **Decupling Logica/Rendering:** Poiché il rendering grafico (Tetris_Draw_Board) richiede tempo, separare la logica di input (nel RIT) dal refresh grafico (gestito nel loop principale o tramite flag) previene il blocco del sistema durante il ridisegno dello schermo.
- **Stabilità del Sistema:** Evita il fenomeno dell'*interrupt storm* che potrebbe verificarsi collegando il joystick rumoroso direttamente alle linee di interrupt esterno.

3 - Timer Hardware e il Sottosistema Audio

Mentre il RIT è dedicato alla scansione periodica degli eventi di gioco (input e gravità), i **Timer General Purpose (Timer0 - Timer3)** dell'LPC1768 sono stati impiegati per compiti che richiedono una precisione temporale più elevata o la generazione di segnali in uscita. In particolare, il progetto sfrutta il **Timer0** come motore per la sintesi audio.

3.1 Differenze Architetturali rispetto al RIT

A differenza del RIT, che dispone di un singolo registro di comparazione, i Timer standard dell'LPC1768 offrono funzionalità avanzate come i **Match Registers** multipli (MR0-MR3) e la capacità di pilotare direttamente i pin di uscita (External Match Output) senza intervento software costante. Questa caratteristica è stata sfruttata per generare le onde quadre necessarie alla riproduzione delle note musicali senza gravare sulla CPU.

3.2 Timer0: Sintesi del Suono

Il modulo audio, implementato in music.c e IRQ_timer.c, utilizza il Timer0 per generare frequenze udibili.

Configurazione e Funzionamento

Il timer è configurato in lib_timer.c per funzionare in modalità **Reset on Match**. Il funzionamento è il seguente:

1. Il *Timer Counter* (TC) viene incrementato ad ogni ciclo di clock periferico (PCLK).
2. Quando il valore del TC eguaglia il valore contenuto nel **Match Register 0 (MR0)**, il timer si resetta automaticamente a zero e genera un interrupt.
3. Contestualmente, il segnale elettrico viene inviato allo speaker per produrre il suono.

Calcolo della Frequenza

La funzione playNote definita in music.c si occupa di convertire le costanti delle note musicali in valori di conteggio per il timer. Poiché il timer determina la durata del semi-periodo (o del periodo intero a seconda della configurazione del toggle), il valore caricato in MR0 determina l'altezza (pitch) della nota.

La frequenza del segnale generato è inversamente proporzionale al valore del registro di match:

$$f_{out} \propto \frac{f_{PCLK}}{MR0}$$

Il codice gestisce dinamicamente il cambio di nota aggiornando il registro MR0 e resettando il timer tramite le chiamate a reset_timer(0) e enable_timer(0).

3.3 Gestione degli Effetti Sonori (SFX)

Il sistema gestisce due tipi di feedback audio definiti in tetris.c:

- **Musica di Sottofondo:** Una melodia ciclica gestita tramite array di frequenze.
- **Effetti Sonori (One-shot):**
    - Play_SFX_LineClear(): Riprodotto quando una linea viene completata (420Hz).

- Play_SFX_Lock(): Riprodotto quando un pezzo tocca il fondo (Nota C5).

L'utilizzo del Timer0 permette di eseguire questi suoni in modo asincrono: una volta impostato il timer, l'hardware si occupa di mantenere l'oscillazione, permettendo alla CPU di continuare a calcolare la logica di gioco e il rendering grafico senza interruzioni udibili.

4 - Interazione Utente: I Pulsanti (External Interrupts)

Mentre il Joystick è gestito tramite polling nel RIT per garantire fluidità di movimento, i comandi critici di gioco come l'avvio della partita (Start) e la caduta istantanea del pezzo (Hard Drop) sono gestiti tramite **Interrupt Esterni (EINT)**. Questa scelta architetturale garantisce che il processore risponda immediatamente a questi input, indipendentemente dallo stato del ciclo di gioco principale.

4.1 Configurazione Hardware e Priorità

L'inizializzazione, definita in lib_button.c, configura i pin P2.10, P2.11 e P2.12 come linee di interrupt esterno. Il controller delle interruzioni (NVIC) è stato configurato con livelli di priorità decrescenti per gestire la concorrenza tra eventi:

1. **EINT2 (KEY2 - Hard Drop):** Priorità **1** (Massima tra i bottoni). Richiede esecuzione immediata per la meccanica di gioco.
2. **EINT1 (KEY1 - Start/Menu):** Priorità **2**. Gestisce il flusso del gioco (Start/Pause).
3. **EINT0 (KEY0):** Priorità **3**.

Inoltre, i registri EXTMODE ed EXTPOLAR sono stati impostati per rilevare i segnali sul **fronte di discesa (Edge Sensitive, Falling Edge)**. Questo significa che l'interrupt viene generato nell'istante esatto in cui l'utente preme il pulsante, non mentre lo rilascia.

4.2 Strategia di Debouncing Ibrida

Uno dei problemi principali dei pulsanti meccanici è il *bouncing* (rimbalzi spuri del segnale). In questo progetto è stata adottata una robusta strategia di debouncing software/hardware definita nelle ISR in IRQ_button.c:

- **Disabilitazione Immediata:** Non appena l'interrupt viene servito, la ISR disabilita la propria linea di interrupt tramite NVIC_DisableIRQ.
- **Gestione "One-Shot":** Questo impedisce che le oscillazioni meccaniche del contatto generino interrupt multipli (e quindi comandi doppi involontari) successivi al primo. La riattivazione dell'interrupt (re-enable) è demandata tipicamente a un timer o alla logica di gioco successiva, garantendo un periodo di "silenzio" che filtra i disturbi.

4.3 Funzionalità Specifiche

Le routine di servizio (IRQ_button.c) implementano logiche specifiche per il controllo del Tetris:

KEY1: Gestione Stato e Generazione Casuale (RNG)

Il pulsante KEY1 (EINT1) funge da pulsante multifunzione per lo Start e il Restart. Una caratteristica dell'implementazione è l'uso del timer hardware come sorgente di entropia per la generazione casuale (Random Number Generation). All'interno della ISR di EINT1, viene eseguito il comando:

```
srand(LPC_RIT->RICOUNTER);
```

Poiché il RICOUNTER scorre velocemente e il momento esatto della pressione del tasto da parte dell'utente è imprevedibile, il valore del contatore in quell'istante fornisce un "seme" (seed) quasi-casuale perfetto per inizializzare la funzione rand(). Questo garantisce che la sequenza dei tetromini generati sia diversa ad ogni partita.

KEY2: Hard Drop

Il pulsante KEY2 (EINT2) è associato all'azione di *Hard Drop* (caduta istantanea). La sua alta priorità assicura che il comando venga recepito istantaneamente, permettendo ai giocatori esperti di velocizzare il gioco senza lag percepibile.

5 - Controllo Direzionale: Il Joystick

Il controllo del movimento dei tetramini (traslazione laterale e rotazione) è affidato al Joystick a 5 posizioni presente sulla scheda LandTiger. A differenza dei pulsanti KEY1 e KEY2 gestiti tramite interrupt asincroni, il Joystick è gestito tramite una strategia di **Polling Periodico** sincronizzata con il timer di sistema.

5.1 Interfaccia Hardware (GPIO)

A livello hardware, le cinque posizioni del joystick (Select, Up, Down, Left, Right) sono collegate alla **Porta 1** del microcontrollore LPC1768, specificamente ai pin P1.25 - P1.29. Nel file lib_joystick.c, la fase di inizializzazione configura questi pin come ingressi digitali (Input) agendo sul registro FIODIR (Fast Input Output Direction). Poiché il joystick chiude il circuito verso massa quando premuto, la logica di lettura è attiva bassa (o gestita tramite resistenze di pull-up interne/esterne a seconda della configurazione elettrica specifica della board).

5.2 Strategia di Gestione: Polling nel RIT

La scelta di non utilizzare gli interrupt esterni per il Joystick è dettata da due fattori principali:

1. **Natura dell'Input:** Il movimento di un pezzo nel Tetris richiede spesso pressioni ripetute o mantenute.
2. **Filtraggio dei Rimbalzi (Debouncing):** I contatti del joystick sono meccanicamente rumorosi.

Invece di attivare una ISR ad ogni micro-vibrazione del contatto, lo stato del joystick viene campionato all'interno della ISR del **RIT (Repetitive Interrupt Timer)** ogni 50ms. Questo intervallo di tempo agisce come un **filtro passa-basso digitale**:

- Vengono ignorati i transitori elettrici più rapidi di 50ms.
- Si garantisce una frequenza di aggiornamento costante (20 Hz) per il movimento, ideale per la reattività del gioco senza sovraccaricare la CPU.

5.3 Logica Funzionale e Mappatura

Il file funct_joystick.c (o la logica integrata in IRQ_RIT.c) si occupa di leggere il registro di stato LPC_GPIO1->FIOPIN e mascherare i bit di interesse per determinare quale direzione è attiva. Le azioni sono mappate direttamente sulle funzioni "Action" del motore di gioco definito in tetris.c:

- **Joystick LEFT (P1.27):** Invoca Action_MoveLeft(). La funzione controlla preventivamente le collisioni; se il movimento porta a una sovrapposizione con blocchi esistenti o con i bordi del campo, la posizione viene ripristinata.
- **Joystick RIGHT (P1.28):** Invoca Action_MoveRight(), con logica di controllo collisioni analoga.
- **Joystick UP (P1.26):** Invoca Action_Rotate(). Questa funzione calcola la rotazione matriciale del pezzo; se la rotazione non è fisicamente possibile (es. adiacenza al muro), viene annullata.

Questa architettura disaccoppiata (Input nel RIT -> Logica in Tetris.c -> Output su LCD) garantisce che la logica di controllo sia robusta e che il rendering grafico avvenga solo quando vi è un effettivo cambiamento di stato, ottimizzando le risorse del bus.

6 - Acquisizione Analogica: Controllo Dinamico della Velocità

Per permettere all'utente di personalizzare la difficoltà di gioco in tempo reale, il sistema sfrutta il potenziometro integrato sulla scheda LandTiger, collegato al **Convertitore Analogico-Digitale (ADC)** del microcontrollore LPC1768. Il valore di tensione letto viene elaborato e mappato per controllare la velocità di caduta dei tetromini.

6.1 Inizializzazione Hardware

Il modulo ADC è configurato nel file lib_adc.c. Il pin P1.31 è stato selezionato tramite il registro PINSEL3 per fungere da ingresso analogico (Canale 5, AD0.5).

La configurazione del registro di controllo ADCR prevede:

- Attivazione dell'alimentazione del blocco ADC (PCONP).
- Impostazione del clock di conversione.
- Abilitazione degli interrupt globali (ADINTEN) per permettere una gestione asincrona del risultato, evitando cicli di attesa attiva (busy-waiting) durante la conversione.

6.2 Gestione dell'Interrupt e Filtraggio del Rumore

Una problematica comune nei sistemi di acquisizione dati è la fluttuazione dei Least Significant Bits (LSB) dovuta al rumore elettrico, che renderebbe instabile la velocità del gioco.

Nella ISR ADC_IRQHandler (file IRQ_adc.c), è stata implementata una strategia di filtraggio software tramite la funzione ADC_CleanNoise:

1. **Lettura:** Il valore raw a 12 bit viene estratto dal registro ADGDR.
2. **Downsampling/Smoothing:** La funzione scarta deliberatamente i 4 bit meno significativi (bit shifting a destra), riducendo la risoluzione effettiva ma eliminando le oscillazioni spurie.

Esempio logico del filtro implementato

```
unsigned short ADC_CleanNoise(unsigned short rawValue, unsigned short nBitDesired){
    unsigned short nShift = 12 - nBitDesired;
    return rawValue >> nShift;
}
```

6.3 Mapping Funzionale: Dal Voltaggio alla Gravità

Il valore filtrato (AD_current) non viene utilizzato direttamente, ma trasformato in un parametro di gioco significativo.

All'interno della ISR, viene calcolata la variabile globale gameSpeedDivider:

$$SpeedDivider = \left(\frac{AD_current \times 4}{15.0} \right) + 1$$

Questa variabile agisce come soglia per il timer della gravità gestito dal RIT: un valore più basso (potenziometro al minimo) indica una frequenza di caduta minore, mentre un valore alto accelera il gioco.

7 - Motore di Gioco e Rendering Grafico Ottimizzato

La logica applicativa del progetto risiede nel modulo tetris.c. Questo componente gestisce lo stato del gioco, la fisica dei pezzi, il rilevamento delle collisioni e, aspetto critico per le prestazioni, l'aggiornamento del display LCD.

7.1 Rappresentazione dello Stato (Strutture Dati)

Il campo di gioco è rappresentato in memoria da una matrice bidimensionale board[20][10] di interi a 16 bit.

- Ogni cella della matrice memorizza il codice colore (formato RGB565) del blocco presente in quella posizione.
- Il valore COL_BACKGROUND (Nero) indica una cella vuota.
- I tetramini sono definiti geometricamente nell'array costante SHAPES come coordinate relative rispetto a un punto pivot centrale (0,0), permettendo una gestione semplificata delle rotazioni matematiche.

7.2 Rilevamento delle Collisioni

La validità di ogni movimento è garantita dalla funzione Check_Collision(Tetromino_t t). Prima di rendere effettivo qualsiasi cambiamento di posizione (dato dal Joystick o dalla gravità), il sistema simula la nuova posizione verificando due condizioni:

1. **Limiti del Campo:** Nessun blocco del tetromino deve uscire dalle coordinate [0, BOARD_COLS-1] e [0, BOARD_ROWS-1].
2. **Sovrapposizione:** Le celle della matrice board corrispondenti alle nuove coordinate devono essere vuote (COL_BACKGROUND). Se una cella è già occupata, il movimento viene annullato.

7.3 Strategia di Rendering: Aggiornamento Parziale

Il display LCD TFT comunica tramite bus parallelo, ma ridisegnare l'intero schermo (320x240 pixel) ad ogni frame (50ms) risulterebbe computazionalmente oneroso, causando sfarfallii (flickering) visibili. Per ovviare a questo problema, è stata implementata una strategia di **Rendering Differenziale** all'interno della funzione Tetris_Draw_Board.

Il sistema utilizza una variabile di stato flag_update_display per determinare la modalità di disegno:

- **Mode 2 (Full Refresh):** Utilizzata solo all'avvio o dopo un Game Over. Ridisegna l'intera griglia, pixel per pixel. È un'operazione lenta ma necessaria per inizializzare la scena.
- **Mode 1 (Partial Refresh - L'Ottimizzazione):** Utilizzata durante il gameplay attivo. Invece di pulire lo schermo, l'algoritmo esegue due passaggi chirurgici:
    1. **Cancellazione:** Utilizza le coordinate della posizione precedente del pezzo (game.prev_current) salvate nel frame precedente per sovrascrivere i vecchi pixel con il colore di sfondo (o ripristinare il contorno).
    2. **Disegno:** Disegna il pezzo nella nuova posizione corrente (game.current).

Questa tecnica riduce drasticamente il numero di operazioni di scrittura sul bus LCD, passando da ~76.000 pixel a poche centinaia per frame, garantendo un'animazione fluida anche ad alte velocità.

7.4 Gestione delle Linee e "Lock" del Pezzo

Quando un pezzo non può più scendere (collisione rilevata sull'asse Y), viene invocata la funzione Lock_Piece.

1. Il pezzo corrente viene "solidificato": le sue coordinate e il suo colore vengono copiati permanentemente nella matrice board.
2. Viene eseguita una scansione delle righe per individuare linee complete.
3. Se una linea è piena, viene eliminata e tutte le righe sovrastanti vengono fatte scorrere verso il basso (copia di memoria riga per riga), incrementando il punteggio.

8 - Funzionalità Avanzate e Implementazione Requisiti Specifici

In conformità con le specifiche del progetto, il motore di gioco base è stato esteso con meccaniche avanzate volte ad aumentare la profondità del gameplay e la difficoltà progressiva. Queste funzionalità includono un sistema di **Power-up** (bonus) e un sistema di **Malus** (penalità), gestiti dinamicamente in base al progresso del giocatore.

8.1 Gestione dei Power-up

Il sistema introduce blocchi speciali che, se eliminati all'interno di una linea completa, attivano effetti benefici per il giocatore. La logica di generazione è gestita dalla funzione Spawn_Powerup, che sostituisce un blocco esistente con uno speciale ogni 5 linee completate.

Sono stati implementati due tipi di Power-up, identificabili dal colore:

1. **Clear Half (Giallo Elettrico - 0xFFE0):**
    - **Effetto:** Rimuove istantaneamente la metà superiore dello stack di pezzi attualmente presenti sulla board.
    - **Implementazione:** La funzione Effect_ClearHalf calcola l'altezza attuale della torre di pezzi. Successivamente, iterando sulla matrice, svuota le righe superiori e compatta quelle rimanenti verso il basso.
    - **Scoring:** Per compensare la perdita di potenziali linee, il sistema assegna un bonus di punteggio calcolato: 800 punti per ogni gruppo di 4 righe rimosse, più 100 punti per le restanti.
2. **Slow Down (Magenta Neon - 0xF81F):**
    - **Effetto:** Rallenta temporaneamente la velocità di caduta dei pezzi, indipendentemente dal valore impostato dal potenziometro.
    - **Implementazione:** La funzione Effect_SlowDown imposta un timer di *override* (game.timer_powerup_slow) a 300 tick (circa 15 secondi). Durante questo periodo, la logica di gravità nel RIT ignora l'accelerazione standard.

L'attivazione avviene nella funzione Lock_Piece: durante la scansione delle linee piene, se la riga contiene un blocco speciale, viene settato un flag (powerup_activated) che innesca l'effetto corrispondente prima della cancellazione della riga.

8.2 Sistema di Malus (Penalità Progressiva)

Per incrementare la difficoltà nelle fasi avanzate della partita, è stata introdotta una meccanica di "Malus" che si attiva ogni 10 linee completate. La funzione Generate_Malus implementa la seguente logica:

1. **Shift Verticale:** Tutte le righe presenti sulla board vengono traslate verso l'alto di una posizione (board[r][c] = board[r+1][c]). Se la riga superiore conteneva pezzi, questi vengono persi (o causano Game Over se toccano il soffitto, a seconda della collisione successiva).
2. **Generazione "Garbage":** L'ultima riga in basso viene riempita con una configurazione casuale di 7 blocchi grigi indestruttibili e 3 spazi vuoti. Questo costringe il giocatore a gestire una superficie di gioco irregolare e riduce lo spazio di manovra disponibile.

8.3 Gestione del Punteggio (High Score)

Il sistema mantiene traccia del punteggio corrente e del miglior punteggio ottenuto (High Score).

- Il punteggio incrementa di base per ogni pezzo piazzato (+10 punti).
- Le linee completate offrono un punteggio moltiplicativo (+100 punti per linea).
- L'High Score è persistente all'interno della sessione di alimentazione: se il giocatore perde (Tetris_GameOver), il record viene confrontato e, se superato, aggiornato per la partita successiva.